

pharo-agentic-browser Scripting API

Headless Multi-Agent Orchestration, Driven by Code

Masashi Umezawa

<https://github.com/mumez/pharo-agentic-browser>

What is the Scripting API?

An optional package that lets you **orchestrate AgenticBrowser topics from code** — no UI interaction required.

- Write a small Smalltalk script; AgenticBrowser creates topics, runs the agents, routes results between them, and blocks until everything completes
- A third interface alongside the Spec UI and Web UI

- **Routine AI workflows** — run the same multi-step workflow daily, or wire it into CI
- **Headless environments** — build and run topics with no display at all
- **Complex multi-agent coordination** — concurrent agents with result routing that's awkward to set up by hand in a UI
- **AI-authored orchestration** — the DSL is simple enough for a coding agent to write and run it itself via `st-eval`

Features

- Just a few builder messages: `seq:`, `para:`, `topicBy:`, `agentBy:`
- Easy enough for a human **or an AI agent** to write and run with a single `eval`
- Results flow automatically between steps — no manual wiring

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t | t prompt: 'Write a one-sentence summary of Pharo.' ]  
  } agentBy: [ :a | a claude ] ].
```

- Coordinate multiple **whole orchestrations**, not just topics within one
- Nest `seq:` / `para:` of orchestrations — and nest groups inside groups
- Enables patterns like **Arena** (same task, different agents, pick the best) or fan-out research merged into one synthesis step

- Orchestrations (and groups) can be **saved and loaded** via Pharo's Fuel serializer
- Save before running to reuse a built configuration later
- Save after running to keep recorded results, then inspect or **resume** from a loaded copy

Installation

Load the Scripting package alongside the core package:

```
Metacello new
  baseline: 'AgenticBrowser';
  repository: 'github://mumez/pharo-agentic-browser:main/src';
  load: 'Scripting'.
```

Optionally, also load the test suite:

```
Metacello new
  baseline: 'AgenticBrowser';
  repository: 'github://mumez/pharo-agentic-browser:main/src';
  load: 'Scripting-Tests'.
```

Script Examples

`runBy:` builds and immediately runs the orchestration, blocking until done:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t title: 'List Pharo features'.  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ]  
    } agentBy: [ :a | a claude ] ].
```

`scriptBy:` builds without running, for later inspection or a `forkRun`.

Each topic's result flows into the next topic's prompt:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ].  
    builder topicBy: [ :t |  
      t prompt: 'Summarize the feature list from the previous step in one sentence.' ]  
  } agentBy: [ :a | a claude ] ].
```

Topics run concurrently; results are combined for the next step:

```
AgenticBrowser runBy: [ :builder |  
  builder para: {  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk language features.' ].  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk development tools.' ]  
  } agentBy: [ :a | a claude ] ].
```

seq: and **para:** steps can be mixed freely — e.g. parallel research → sequential synthesis.

Run two candidates with different agents, then pick the winner:

```
AgenticBrowser groupRunBy: [ :groupBuilder |
  groupBuilder para: {
    groupBuilder orchestrationBy: [ :b |
      b seq: { b topicBy: [ :t | t prompt: 'Implement feature XXX.'. t goal: 'all tests pass' ] }
      agentBy: [ :a | a claude ] ].
    groupBuilder orchestrationBy: [ :b |
      b seq: { b topicBy: [ :t | t prompt: 'Implement feature XXX.'. t goal: 'all tests pass' ] }
      agentBy: [ :a | a codex ] ]
  }.
  groupBuilder singleTopicBy: [ :t | t prompt: 'Pick the best implementation above and open a PR.' ]
  agentBy: [ :a | a kilo ]
].
```

Execution Variations

Synchronous — `runBy:` / `script run` blocks until everything completes.

Background — for long-running orchestrations:

```
script forkRunThen: [ :orc | Transcript crShow: 'Done: ', orc result ].  
"... later, if needed:"  
script isRunning.
```

`forkRun` is a shorthand for `forkRunThen: [ :orc | ]`.

While a background orchestration runs, open the **Spec UI** or **Web UI** to watch it live — think messages, permission confirmations, and topic progress all show up there.

Cancel a running orchestration:

```
script terminate.
```

Resume after a step times out (continues from the first incomplete step, reusing recorded results):

```
script resume.
```

Save & Load

Persist an orchestration (or group) via Fuel — before or after running:

```
script save.                                     "-> <sharedDirectoryPath>/<name>.fuel"
script saveTo: '/path/to/my-script.fuel' asFileReference.

loaded := AbTopicOrchestration loadFrom: '/path/to/my-script.fuel'.
loaded result.                                   "results from the saved run"
loaded resume.                                  "continue if the saved run stopped early"
```

A saved orchestration can also be loaded straight into a new group with

```
orchestrationLoadFrom: .
```

Settings

Setting	Default	Scope
<code>orchestrationStepWaitTimeoutSeconds</code>	900 s	Per topic step
<code>orchestrationGroupItemWaitTimeoutSeconds</code>	3600 s	Per group item running in <code>para:</code>

Adjust globally or per orchestration/group:

```
script settings orchestrationStepWaitTimeoutSeconds: 1800.
```

Setting	Default	Effect
<code>lingerOrchestrationTopicsAfterRun</code>	<code>false</code>	Whether topics stay in the Topic Manager after the orchestration finishes

- `false` — topics are removed from the Topic Manager once the run completes
- `true` — topics remain, so you can review the whole run's progress from the UI afterward

```
script settings lingerOrchestrationTopicsAfterRun: true.
```

Set it to `true` when you want to go back through the Spec UI or Web UI later and retrace how the result was reached.

Summary

The **Scripting API** brings headless, code-driven orchestration to AgenticBrowser:

- **Simple DSL** — `seq:` / `para:` / `topicBy:` / `agentBy:`, easy for humans and AI agents alike
- **Orchestration Groups** — compose whole workflows in parallel, in sequence, or nested
- **Result routing** — automatic, no manual wiring between steps
- **Flexible execution** — synchronous or background, with cancel and resume
- **Fuel persistence** — save and reload orchestrations and their results

Feedback and contributions are welcome!

<https://github.com/mumez/pharo-agentic-browser>